involves randomly choosing actions, and a high temperature involves random changes as well. They're quite analogous.

Modeling the behavior of electron spins may seem unimportant, but the same basic modeling technique used for electrons can be used to solve problems in genetics, finance, economics, botany, and sociology, among others. It also happens to be one of the simplest ways to test out MARL, so that's our main motivation here.

The only thing we need to do to create an Ising model is to create a grid of binary digits where 0 represents spin-down and 1 represents spin-up. This grid could be of any dimensions. We could have a one-dimensional grid (a vector), a two-dimensional grid (a matrix), or some high-order tensor.

Over the next few code listings we will first solve the 1D Ising model, since it is so easy that we don't need to use any fancy mechanisms like experience replay or distributed algorithms. We won't even use PyTorch's built-in optimizers—we will write out the gradient descent manually in just a few lines of code. In listing 9.3 we'll define some functions to create the electron grid.

Listing 9.3. 1D Ising model: Create the grid and produce rewards

```
import numpy as np
import torch
from matplotlib import pyplot as plt

def init_grid(size=(10,)):
    grid = torch.randn(*size)
    grid[grid > 0] = 1
    grid[grid <= 0] = 0
    grid = grid.byte()    1
    return grid

def get_reward(s,a):      2
    r = -1
    for i in s:
        if i == a:
            r += 0.9
    r *= 2.
    return r
```